

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère un dictionnaire `depenses` dont les clés sont des catégories de dépenses (chaînes de caractères) et les valeurs sont les montants correspondants (nombres réels).

Exemple :

```
depenses = {"courses": 85.5, "transport": 32.0, "loyer": 520.0, "cafe": 9.5}
```

1. Écrire une fonction `top_categorie(depenses)` qui renvoie un tuple (`cat`, `montant`) correspondant à la catégorie dont le montant est maximal.

En cas d'égalité entre plusieurs catégories, on choisira celle dont le nom est le plus petit selon l'ordre alphabétique. Si le dictionnaire `depenses` est vide, la fonction renverra le tuple `("", 0)`.

2. Écrire une fonction `rapport(depenses, budget)` qui prend en entrée le dictionnaire `depenses` et un dictionnaire `budget` ayant les mêmes clés que `depenses` et qui renvoie un **nouveau** dictionnaire `R` tel que pour toute catégorie `cat` :

$$R[cat] = depenses[cat] - b,$$

où b est égal à `budget[cat]` si `cat` appartient au dictionnaire `budget`, et à 0 sinon.

Exemple :

```
budget = {"courses": 80.0, "transport": 40.0, "loyer": 520.0}
# R["courses"] = 5.5
# R["transport"] = -8.0
# R["loyer"] = 0.0
# R["cafe"] = 9.5
```

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère un flux de commandes représenté par une liste de chaînes de caractères correspondant à des plats, précédés du symbole “-” pour indiquer une annulation de commande. Le but est de séparer ces commandes en deux **files** : une file contenant les commandes à préparer et une file contenant les commandes annulées.

1. Écrire une fonction Python `enfiler(f, plat)` qui ajoute la chaîne de caractères `plat` dans la file `f`.
2. Écrire une fonction Python `traiter_commandes(commandes)` qui récupère les commandes dans la liste `commandes` une par une, et :
 - ajoute la commande dans la file `a_preparer` si elle ne commence pas par “-”,
 - ajoute la commande (sans le “-”) dans la file `annulees` si elle commence par “-”.

Vérifier que toutes les commandes ont été traitées et que la liste `commandes` est vide au fil du processus.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère la suite (u_n) définie par :

$$\begin{cases} u_0 = 1 \\ u_{n+1} = -0.5u_n + 3 \end{cases} \text{ pour tout } n \geq 0.$$

Les termes de cette suite peuvent être calculés progressivement à partir de la valeur initiale u_0 .

1. Écrire une fonction `terme_n_suite` qui prend en argument un entier n et qui retourne la valeur du terme u_n de la suite. Tester cette fonction pour $n = 5$ et $n = 10$.
2. Écrire une fonction qui renvoie la somme des N premières éléments de la suite. Tester cette fonction pour $N = 10$.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On appelle décomposition d'un entier naturel n un couples (i, j) tel que $i^2 + j^2 = n$. Nous allons construire une matrice de dimension $m \times m$ qui contient dans la case (i, j) l'entier n tel que $i^2 + j^2 = n$.

1. Écrire une fonction qui prend en entrée un entier m et qui renvoie la matrice des décompositions, pour tous les entiers $1 \leq i, j \leq m$.

Exemple: Pour $m = 3$ on a :

$$\begin{pmatrix} 2 & 5 & 10 \\ 5 & 4 & 13 \\ 10 & 13 & 18 \end{pmatrix}$$

2. Écrire une fonction qui prend en entrée une matrice de décompositions et qui renvoie la liste de nombres entiers dont la décomposition se trouve dans la matrice.

Exemple: Pour la matrice ci-dessus on obtient la liste $[2, 5, 10, 4, 13, 18]$.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On souhaite simuler une file d'attente (par exemple à un guichet). En Python, la file d'attente sera représentée par une liste de chaînes de caractères.

1. Écrire une fonction `ajouter_client(file_attente, nom)` qui prend en paramètre une liste `file_attente` et une chaîne de caractères `nom` et ajoute le client `nom` dans la file.
2. Écrire une fonction `servir_client(file_attente)` qui prend en paramètre une liste `file_attente` et retire un client de la file. Si la file est vide, on affiche le message "**Aucun client à servir**" et on renvoie `None`.
3. Écrire un programme principal qui :
 - initialise une file d'attente vide,
 - demande à l'utilisateur combien de clients arrivent successivement,
 - pour chaque client, demande son nom et appelle `ajouter_client`,
 - appelle ensuite `servir_client` une première fois et affiche le client servi,
 - affiche enfin le contenu restant de la file d'attente.

Remarques étudiant :
Remarques correcteur :